

Connected Neighbours "Voisins, services et bonne humeur" Merci à Baptiste Laurent, un de vos prédécesseurs en 3AL

Objectif

L'objectif de ce projet est de construire une plateforme collaborative sécurisée et extensible de quartiers permettant aux habitants d'échanger des services, de signer des documents numériques, de participer à des événements et de communiquer via une messagerie multimédia.

Principales fonctionnalités Web

Modélisation géographique d'un quartier

La plateforme doit permettre à l'administrateur de définir un quartier géographiquement, à l'aide d'un outil de dessin. Prévoir les problèmes de limites.

Services entre voisins

La plateforme doit, grâce à un système de petites annonces, permettre d'offrir ou demander des services, gratuits ou non. Dans le cas où les services seraient payants, il sera nécessaire de créer un système de points (par exemple, faire du babysitting pendant 3 heures pourrait donner quatre points, donner un cours particulier d'une heure en apportant deux etc ...). Chaque service payant donnera lieu à la création d'un contrat obligatoire.

Documents et signatures en ligne

De nombreux documents, dont les contrats, peuvent être échangés entre les habitants du quartier. Il sera donc important de prévoir une fonction d'import de PDF avec zones de signature / initiales, de sécuriser les signatures numériques et de les archiver.

Événements et activités

La vie de quartier est rythmée par des activités communautaires (soirées, ateliers, collectes de fond ...), que les habitants pourront créer, gérer, consulter (avec swipe pour indiquer l'intérêt).

Des suggestions automatiques pourront être proposées grâce à Neo4j (en se basant sur des interactions passées et événements qui intéressent l'utilisateur).

Messagerie multimédia sécurisée

Quoi de mieux que de discuter ensemble ! Prévoir un chat texte, avec possiblement des photos ou des vocaux, ainsi que des appels vidéos (un bonus). Les présences online/offline seront indiquées en temps réel.

Votes

Il est quelquefois nécessaire de voter sur un élément de la vie quotidienne du quartier. Prévoir un système de gestion paramétrable et extensible.

Noter que bien entendu, l'application est multilingue.

Cette liste n'est pas exhaustive et il doit être bien sur possible de rajouter les modules qui vous paraîtraient intéressants, en prévoyant un système qui permette de le faire sans modifier le code existant.

Client Java Desktop

Pour compléter l'application précédente, l'administrateur doit disposer d'une application desktop Java permettant de gérer les incidents et alertes dans le quartier, signalés par les voisins sur l'application Web, ainsi que des statistiques sur les participations des voisins dans le quartier. Toutes les données doivent être enregistrées dans une base de données locale embarquée, pour fonctionner même si Internet est en panne.

En mode offline-first, la consultation des incidents et statistiques est possible sur les données déjà synchronisées. L'ajout de données est également possible.

Il est important de prévoir une synchronisation automatique dès qu'une connexion est disponible (en résolvant éventuellement les conflits).

De plus, l'application assure la gestion de :

- Plugins : export statistiques, analyse sociale, calendrier local.
- Thèmes : personnalisation complète (couleurs, polices, disposition).
- Mises à jour automatiques de l'application via serveur central.

L'application doit pouvoir être désinstallée depuis l'interface utilisateur.

L'application devra être rendu sous la forme d'un fichier .jar.

Caractéristiques techniques

Sécurité

- MFA obligatoire pour toutes les actions sensibles (connexion, modification du mot de passe/e-mail / téléphone, signature de documents).
- SSO (Single Sign-On) entre le site web et l'application Java.
- Gestion des rôles : habitants, modérateurs, administrateurs.

RGPD

Toutes les fonctionnalités liées aux données personnelles (authentification, messagerie, signatures de documents, stockage des interactions) devront être conçues dans le respect du référentiel RGPD, incluant le droit d'accès, de rectification, de suppression des données et d'export des données.

Conteneurisation

Les différentes applications doivent être conteneurisées et séparées en plusieurs environnements et des tests (unitaires, E2E, intégration, ...) devront être implémentés.

Persistance et moteur de recommandations

Plusieurs BDD sont demandées :

- MongoDB : stockage des documents (contrats, signatures), événements, messages.
- Neo4j : graphe social des interactions (qui a aidé qui, participation commune aux événements).
Sert de base au moteur de recommandations (voisins fiables, événements pertinents, services compatibles).
- Il est d'ailleurs demandé de créer un langage d'interrogation maison (via lex/yacc) pour manipuler les documents MongoDB.

Technologies imposées

- Back-end : Node.js + Express
- SGBD : MongoDB + Neo4j + Base locale (SQLite/H2) pour l'application Java
- Front-end Web : React (site utilisateur + back office)
- Client lourd : Java et JavaFX

Documents demandés

- Tous les schémas nécessaires pour expliquer l'organisation de vos applications (interactions entre les différentes briques applicatives -bases, API externe, architecture des conteneurs, ...)
- Modélisation des bases de données.
- Descriptif des fonctionnalités (API, front, Java, ...).
- Documentation de type Swagger pour l'Api.
- Explication des tests.

Exemple d'architecture

